

CUDA in Containers & CI Validation

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India



What you will be able to do after this session

- Sketch the CUDA execution model: host vs device, kernels, grids of blocks of threads.
- Compile and run a vector-add kernel inside a devel container — hands-on.
- Use a 3-level probe (driver → toolkit → framework) to localize GPU stack failures.
- Design a CI pipeline: build → smoke test → scan → push, with GPU-aware caveats.

Why touch CUDA this course?

1 Ground truth

A 30-line kernel that compiles and runs proves driver, toolkit, and GPU agree — independent of any framework.

2 Vocabulary

Logs and errors speak CUDA: 'invalid device function', 'sm_100', 'illegal memory access'. You need to read them.

3 Arch targets

Wheels and kernels are compiled per GPU architecture (Blackwell = sm_100). Mismatches surface as runtime errors.

The CUDA execution model in one slide

HOST (CPU)		DEVICE (GPU)
-----		-----
allocate + copy data	----->	global memory
launch kernel<<<blocks,threads>>>		grid = blocks x threads
		each thread: i = blockIdx.x * blockDim.x + threadIdx.x
copy results back	<-----	global memory

- A kernel is a function run by thousands of threads at once, each on its own index.
- The <<<blocks, threads>>> launch syntax sizes the grid to cover the data.
- Frameworks like PyTorch do exactly this under every tensor op — just pre-written and tuned.

Thread Hierarchy — Grid, Block, Warp, Thread

Grid — 1 per kernel launch

Block (up to 1024 threads) — Runs on one SM; shares `__shared__` memory

Warp (32 threads) — The hardware scheduling unit — SIMT execution

Thread — One CUDA core; has private registers

Key rule: threads in a warp execute the same instruction simultaneously. Divergent branches (if/else on `threadIdx`) serialize — one path at a time. Minimize divergence.

```
// Launch config: 1D grid of blocks, 256 threads each
int blocks = (N + 255) / 256;
kernel<<<blocks, 256>>>(d_a, d_b, d_c, N); // gridDim.x=blocks, blockDim.x=256
```

GPU Memory Hierarchy — Where Data Lives and How Fast

Registers	~10 TB/s	Per thread — private, fastest, limited (~255 per thread)
Shared Memory (__shared__)	~10 TB/s	Per block — on-chip, programmer-managed scratchpad
L2 Cache	~8 TB/s	Chip-wide — automatic, no programmer control
HBM3e (Global Memory)	~8 TB/s	All threads — main VRAM, cudaMalloc target (B200: 8 TB/s)
PCIe / System RAM	~32–64 GB/s	Host ↔ Device transfers — 125–250× slower than HBM

Rule: keep hot data in shared memory, minimise PCIe transfers, coalesce global memory accesses.

Unified Memory vs Explicit Transfers — Trade-offs

cudaMallocManaged (Unified Memory)

Single pointer accessible from host and device. CUDA runtime migrates pages on demand. Simple to write. Under the hood: same PCIe transfers, but triggered by page faults — unpredictable timing, hard to overlap with compute.

When to use Unified Memory

Prototyping and teaching (vector_add.cu). Irregular access patterns where you don't know what to prefetch. CUDA 12+ prefetch hints (cudaMemPrefetchAsync) close much of the gap.

```
// Unified Memory - simple
float *a;
cudaMallocManaged(&a, N*sizeof(float));
kernel<<<blocks,256>>>(a, N);
cudaDeviceSynchronize(); // wait before CPU
reads
```

```
// Explicit - predictable, overlap-friendly
float *d_a, *h_a = new float[N];
cudaMalloc(&d_a, N*sizeof(float));
// Pin host buffer for max PCIe BW:
float *h_pinned;
cudaMallocHost(&h_pinned, N*sizeof(float));
cudaMemcpy(d_a, h_pinned, N*sizeof(float),
           cudaMemcpyHostToDevice);
kernel<<<blocks,256>>>(d_a, N);
cudaMemcpy(h_pinned, d_a, N*sizeof(float),
           cudaMemcpyDeviceToHost);
cudaFree(d_a); cudaFreeHost(h_pinned);
```

Occupancy — Keeping the SM Busy

What occupancy means

The ratio of active warps to maximum warps an SM can host. Higher occupancy = more warps ready to hide latency when one warp stalls waiting for memory.

Target: $\geq 50\%$ for memory-bound kernels.

What limits occupancy

Register usage per thread (cap: ~ 65536 per SM). Shared memory per block (cap: 48–228 KB per SM depending on GPU). Block size (must be multiple of 32; 128 or 256 usually best).

```
// Query occupancy at runtime
int minGridSize, blockSize;
cudaOccupancyMaxPotentialBlockSize(
    &minGridSize, &blockSize, myKernel, 0, 0);

// Or check register usage:
// nvcc --ptxas-options=-v kernel.cu
// ptxas info: Used 32 registers, 0 bytes smem

// Compile-time limit (rarely needed):
__global__ void __launch_bounds__(256, 4)
myKernel(...) { ... }
// max 256 threads/block, min 4 blocks/SM
```

Rule of thumb

Start with blockDim=256. If register spill warnings appear, reduce to 128. Profile with Nsight before micro-optimising.

CUDA Streams — Overlapping Compute and Data Transfer

What a stream is

A sequence of CUDA operations that execute in order. Multiple streams can run concurrently. Default stream (stream 0) is synchronous with all other streams.

Why it matters

Without streams: H2D transfer → kernel → D2H transfer. All sequential. With 2 streams: overlap H2D of batch N+1 while kernel processes batch N. Theoretical 2× throughput on data-parallel workloads. PyTorch uses streams internally for this.

```
cudaStream_t s1, s2;
cudaStreamCreate(&s1);
cudaStreamCreate(&s2);

// Overlap: transfer next batch while computing current
cudaMemcpyAsync(d_in1, h_in1, sz, H2D, s1);
cudaMemcpyAsync(d_in2, h_in2, sz, H2D, s2);

kernel<<<grid, block, 0, s1>>>(d_in1, d_out1);
kernel<<<grid, block, 0, s2>>>(d_in2, d_out2);

cudaMemcpyAsync(h_out1, d_out1, sz, D2H, s1);
cudaMemcpyAsync(h_out2, d_out2, sz, D2H, s2);

cudaStreamSynchronize(s1);
cudaStreamSynchronize(s2);
cudaStreamDestroy(s1); cudaStreamDestroy(s2);
```

CUDA Error Handling — Why Silent Failures Are the Worst Kind

The silent failure problem

Kernel launches NEVER return an error code directly — they are asynchronous. An OOM or invalid memory access inside a kernel only becomes visible when you next call a synchronizing CUDA function. This can be lines later.

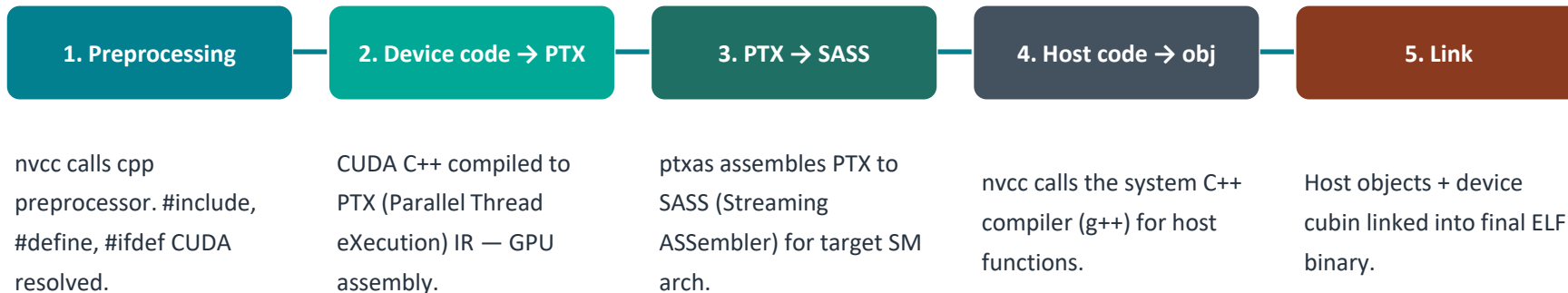
The cuda-memcheck / compute-sanitizer approach

Run compute-sanitizer --tool memcheck ./vector_add to catch out-of-bounds, unaligned, and use-after-free errors at run time. compute-sanitizer is the successor to cuda-memcheck. Adds ~10× overhead — only for debugging.

```
// Helper macro — use this everywhere:
#define CUDA_CHECK(err) do { \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error %s at %s:%d\n", \
            cudaGetErrorString(err), __FILE__, \
            __LINE__); \
        exit(1); \
    } \
} while(0)
```

```
// Usage:
CUDA_CHECK(cudaMalloc(&d_a, N*sizeof(float)));
CUDA_CHECK(cudaMemcpy(d_a, h_a, N*sizeof(float), \
H2D));
kernel<<<grid, block>>>(d_a, N);
CUDA_CHECK(cudaGetLastError()); // catch
launch errors
CUDA_CHECK(cudaDeviceSynchronize()); // flush
async errors
```

nvcc Compilation Pipeline — .cu to Binary



```
# Compile for current GPU (requires devel image, not runtime):
nvcc -arch=native vector_add.cu -o vector_add

# Generate PTX for any SM (virtual architecture):
nvcc -arch=compute_80 -code=sm_80,sm_90 add.cu -o add

# Inspect PTX output (readable GPU IR):
nvcc -ptx add.cu -o add.ptx && cat add.ptx

# Makefile pattern used in demo (falls back sm_86 if sm_100 unavailable):
ARCH := $(shell nvcc --query-gpu-name 2>/dev/null | grep -q "B200" && echo sm_100 || echo sm_86)
NVCCFLAGS = -arch=$(ARCH) -O2
```

devel vs runtime — What Each Image Contains

Component	runtime	devel
CUDA runtime (libcudart)	✓	✓
cuBLAS, cuDNN, cuSPARSE	✓ (separate image tags)	✓
NCCL	✗ (separate image)	✓
nvcc compiler	✗	✓
CUDA headers (.h files)	✗	✓
Static libraries (.a)	✗	✓
ptxas, fatbinary tools	✗	✓
Image size (approx, CUDA 13)	~3 GB	~8 GB

Multi-stage pattern: compile in devel → COPY binary → run in runtime. Ship the smallest image that can actually run your model.

Lab 1 — A vector-add kernel (the code)

```
// vector_add.cu
#include <stdio>

__global__ void add(const float* a, const float* b, float* c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) c[i] = a[i] + b[i];
}

int main() {
    int n = 1 << 20;  float *a, *b, *c;                // 1M floats
    cudaMallocManaged(&a, n * 4); cudaMallocManaged(&b, n * 4);
    cudaMallocManaged(&c, n * 4);
    for (int i = 0; i < n; i++) { a[i] = 1.0f; b[i] = 2.0f; }
    add<<<(n + 255) / 256, 256>>>(a, b, c, n);
    cudaDeviceSynchronize();
    printf("c[0]=%.1f  c[n-1]=%.1f\n", c[0], c[n - 1]);  // 3.0 3.0
}
```

Lab 1 — Compile and run it in a container

```
# devel image = nvcc included. Mount the code, compile, run.
docker run --rm --gpus all -v "$PWD":/src -w /src \
  nvidia/cuda:13.0.0-devel-ubuntu24.04 bash -c '
    nvcc -arch=native vector_add.cu -o vector_add && ./vector_add'

# Expected:  c[0]=3.0  c[n-1]=3.0

# Cross-compile for a specific arch (e.g. Blackwell):
nvcc -gencode arch=compute_100,code=sm_100 vector_add.cu -o vector_add
```

- If this prints 3.0, the entire stack — driver, runtime, toolkit, GPU — is healthy.
- Keep this as your permanent 'is it the cluster or my code?' probe.

Localizing failures: the 3-level probe

- 1 Driver level: `nvidia-smi` inside the container — GPU visible at all?
- 2 Toolkit level: the vector-add probe — can CUDA code compile and execute?
- 3 Framework level: `python -c 'import torch; print(torch.cuda.is_available())'`

Whichever level fails first names the layer to fix. Never debug level 3 before levels 1–2 pass.

CI for AI images: rebuild trust on every commit

- Environments rot quietly: a base-tag update or a yanked wheel breaks Tuesday's build of Monday's code.
- CI makes the image a tested artifact: every commit rebuilds it and proves it still works.
- The pipeline is also documentation — the workflow file IS the build procedure.
- GPU caveat: most hosted runners are CPU-only; smoke tests must degrade gracefully (import + CPU ops), with full GPU tests on self-hosted runners.

Anatomy of an image pipeline

1 Build

docker build with BuildKit + registry cache.

2 Smoke test

Run the image; imports, versions, tiny op. Under a minute.

3 Scan

CVE scan (e.g. trivy); fail on critical findings.

4 Push

Tag with git SHA + semver; push only after 1–3 pass.

Smoke tests that catch real breakage

```
# smoke_test.py - runs in CI (CPU) and on GPU
import torch, transformers, sys
print('torch', torch.__version__, '| transformers',
      transformers.__version__)
x = torch.randn(64, 64) @ torch.randn(64, 64)      # CPU math works
if torch.cuda.is_available():                       # GPU tier (self-
hosted)
    y = x.cuda() @ x.cuda(); torch.cuda.synchronize()
    print('gpu ok:', torch.cuda.get_device_name(0))
else:
    print('cpu-only runner: gpu checks skipped')
sys.exit(0)
```

- Catches: missing libs, broken wheels, version mismatches, CUDA-less wheels — in seconds.
- Same script, two tiers: CPU assertions everywhere, GPU assertions where hardware exists.